

Smart Task Manager

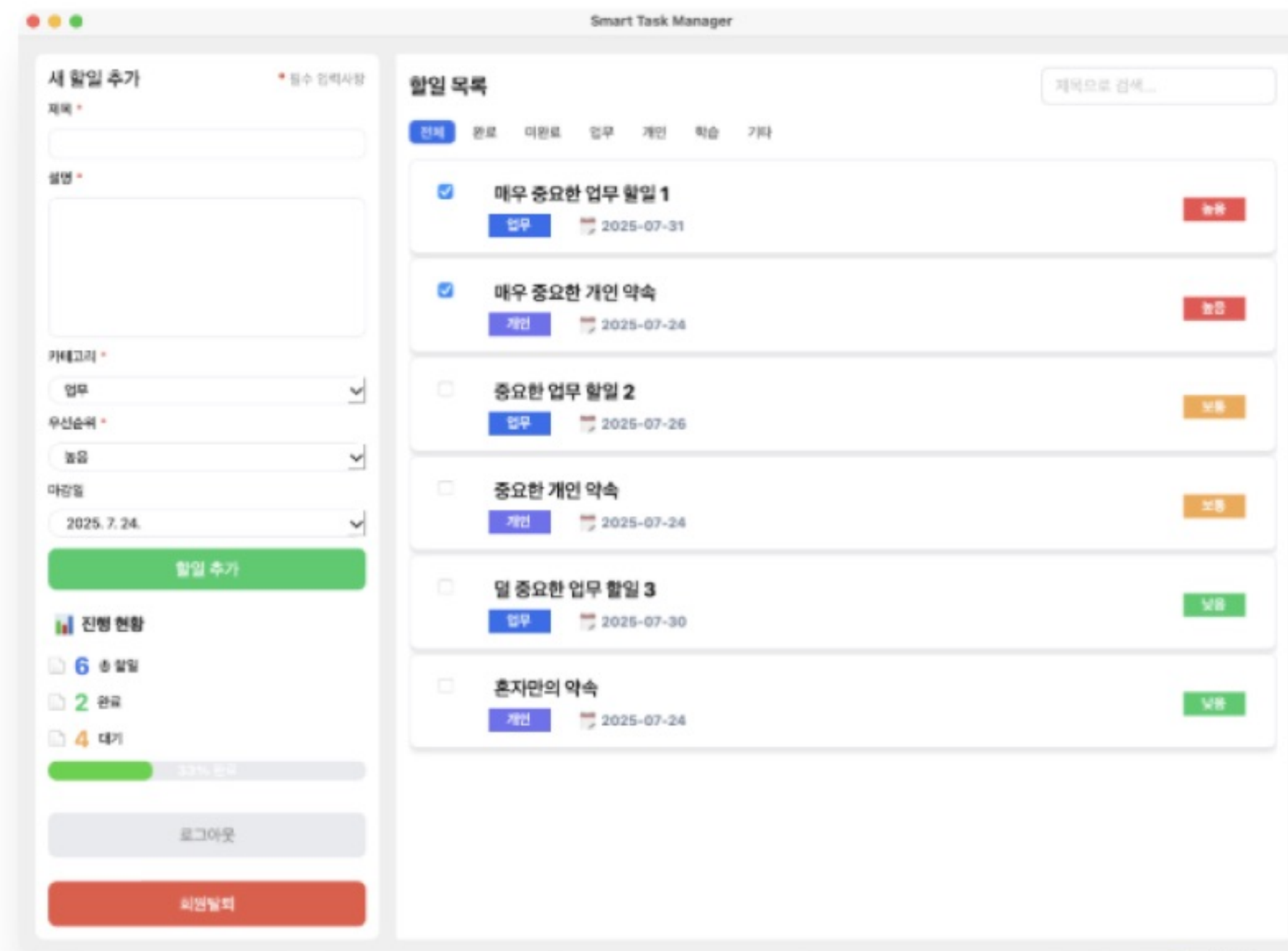
현대인을 위한 스마트한 할일 관리 애플리케이션

개인과 팀의 업무 효율을 높이기 위한 데스크톱 기반 태스크 관리 앱

✓ 직관적인 사용자 인터페이스

👤 개인 맞춤형

📊 통계 시각화



기획 배경



출처: 게티이미지뱅크

주요 기능



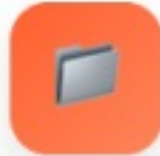
사용자 인증

- ✓ JWT 기반 토큰 인증 시스템
- ✓ bcrypt로 비밀번호 암호화
- ✓ 회원가입, 로그인, 로그아웃, 회원탈퇴 지원



할일 관리

- ✓ 할일 생성(Create) 및 조회(Read)
- ✓ 내용 및 상태 수정(Update)
- ✓ 불필요한 할일 삭제>Delete)
- ✓ 완료 상태 토글(Toggle) 기능



카테고리 분류 & 우선순위

- ✓ 4가지 카테고리 분류 (업무, 개인, 학습, 기타)
 - ✓ 3단계 우선순위 설정
- 우선순위 레벨

높음

중간

낮음



검색 및 필터링

- ✓ 키워드 기반 할일 검색
- ✓ 카테고리별 필터링
- ✓ 우선순위 기준 정렬
- ✓ 완료/미완료 상태별 필터링



통계 대시보드 시각화

- ✓ 할일 완료율 시각화

기술 스택

구분	기술	역할 및 선정 이유
 프론트엔드	PyQt6	데스크톱 UI 개발을 위한 파이썬 바인딩 라이브러리입니다. 네이티브 애플리케이션의 성능과 안정성을 제공합니다.
 백엔드	FastAPI	현대적이고 빠른 고성능 웹 프레임워크입니다. 비동기 처리를 지원하여 API 응답 속도를 높이고, 자동 API 문서 생성 기능으로 개발 및 협업 생산성 향상 등의 장점이 있습니다.
 데이터베이스	MariaDB	널리 사용되는 오픈소스 관계형 데이터베이스 관리 시스템(RDBMS)입니다. MySQL과 호환성이 높고 안정적인 성능을 보장하여 데이터의 정확성과 신뢰성 확보가 가능합니다.
 배포	Docker	애플리케이션을 컨테이너 환경으로 패키징하여 환경 불일치 없이 실행할 수 있습니다. <code>docker-compose</code> 를 이용해 데이터베이스, 백엔드, 프론트엔드를 통합적으로 구성하고 손쉽게 실행 및 배포가 가능합니다.

주요 화면

로그인 회원가입

로그인 화면

Smart Task Manager


로그인
Smart Task Manager에 오신 것을 환영합니다.

아이디 *

example@naver.com

비밀번호 *

password

로그인

[계정이 있으신가요?](#) [회원가입](#)

 JWT 기반 인증으로 안전한 로그인, bcrypt로 암호화된 비밀번호

회원가입 화면

Smart Task Manager


회원가입
Smart Task Manager에 오신 것을 환영합니다.

아이디 *

example@naver.com

비밀번호 *

password

이름 *

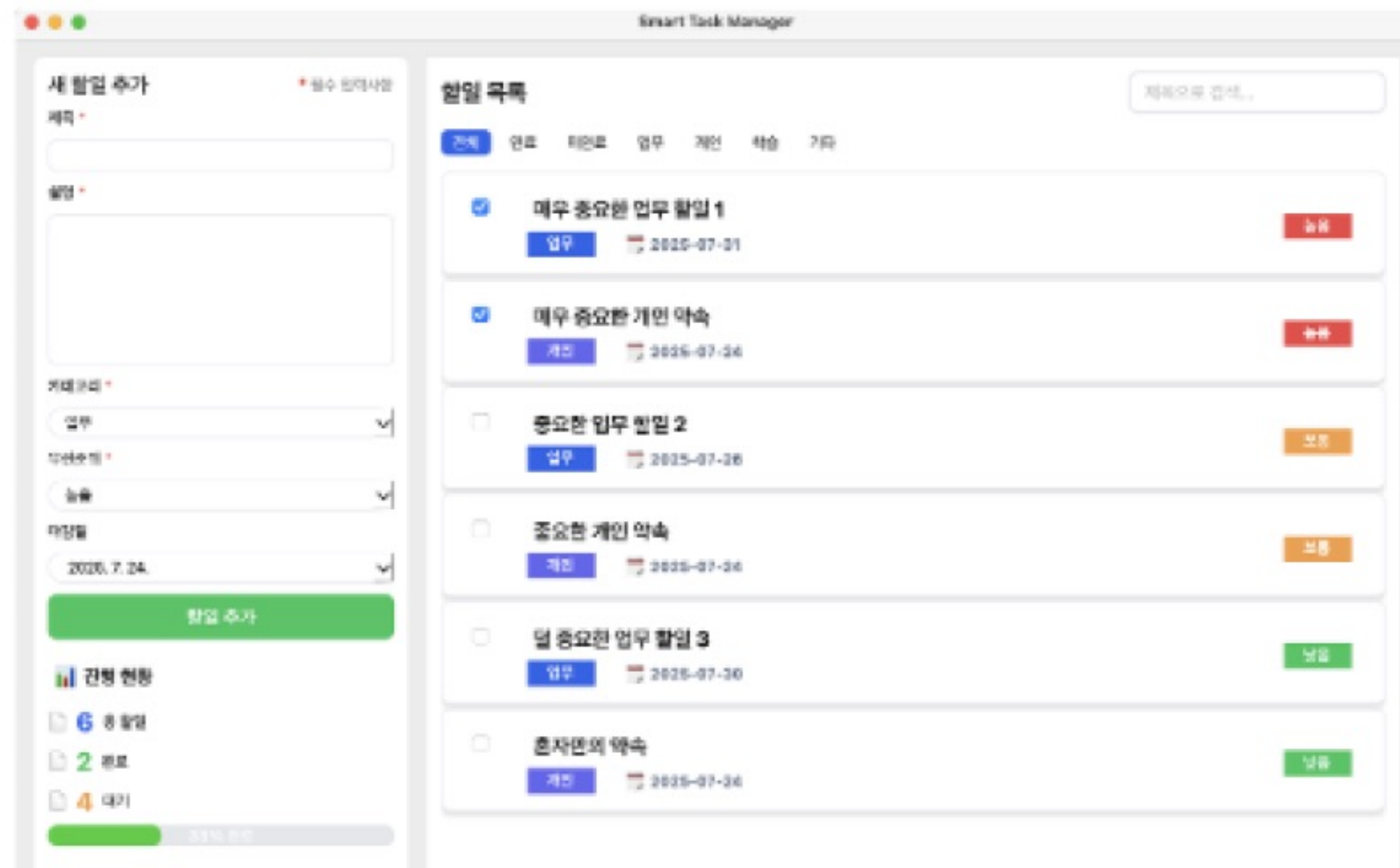
홍길동

회원가입

[이미 계정이 있으신가요?](#) [로그인](#)

메인 할일 화면 및 통계 대시보드

☰ 메인 할일 화면



✓ 할일 생성, 수정, 삭제, 완료 토글 기능 제공

📊 통계 대시보드

📊 진행 현황

📄 6 총 할일

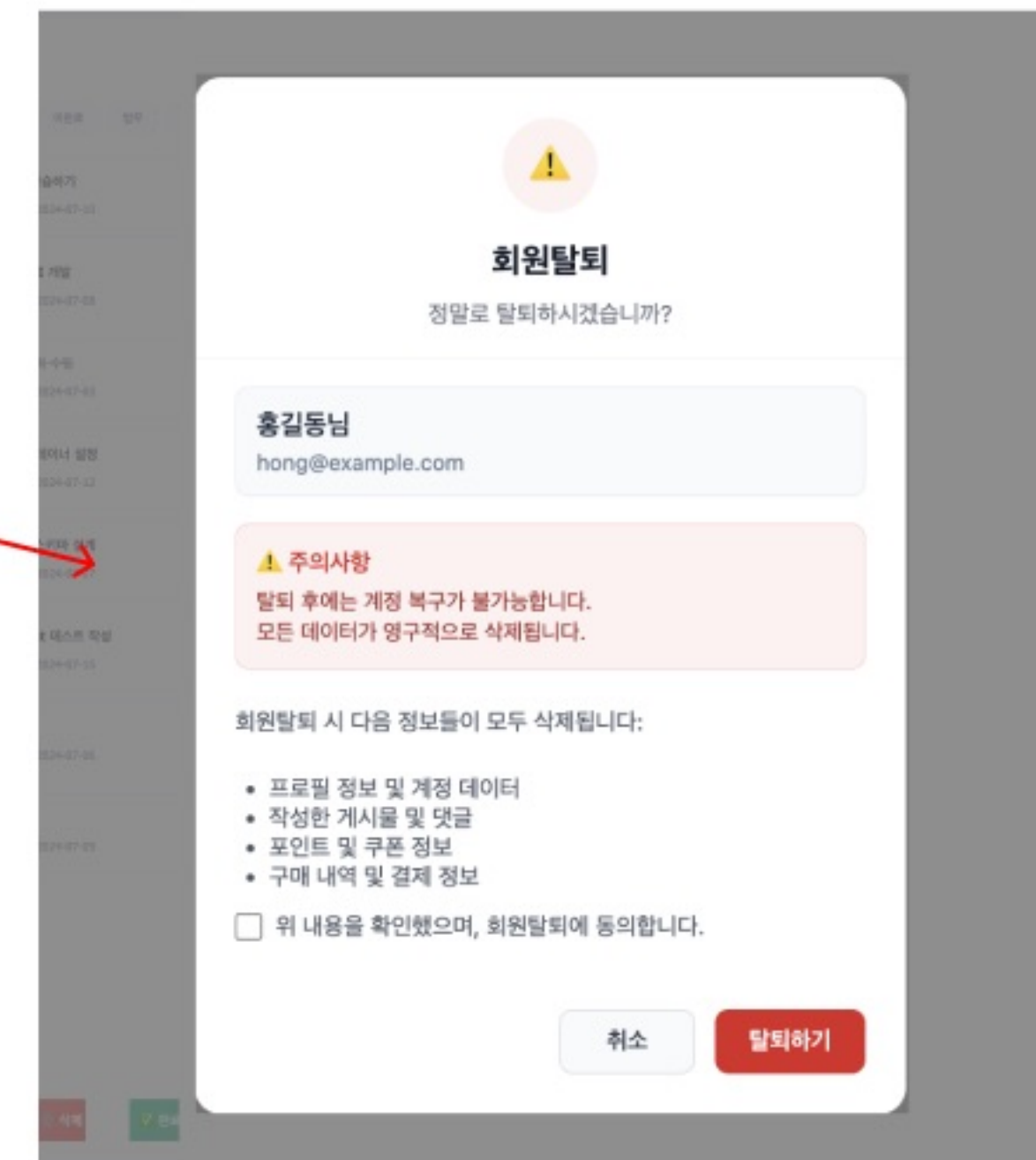
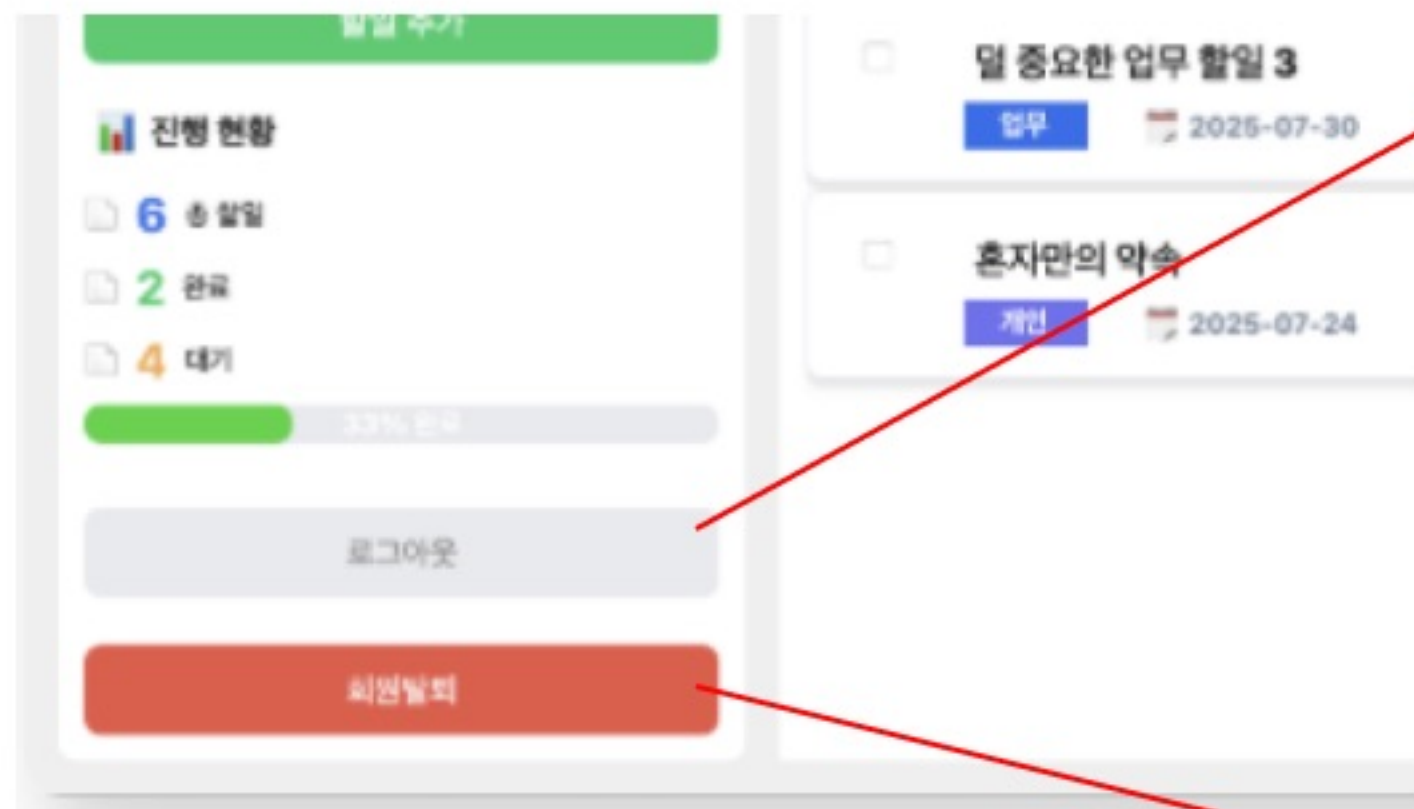
📄 2 완료

📄 4 대기

33% 완료

📊 전체 및 카테고리별 완료율을 시각적으로 분석

로그아웃, 회원탈퇴



시연



인증 & 할일 기능

👤 사용자 인증 기능

JWT + bcrypt를 활용한 안전한 사용자 인증 시스템



회원가입

간단한 정보 입력으로 계정 생성

이메일, 비밀번호 입력 → bcrypt 암호화 → DB 저장



로그인

계정 인증 및 JWT 토큰 발급

비밀번호 검증 → JWT 생성 → 클라이언트 저장



로그아웃

JWT 토큰 제거 → 로그인 화면으로 리디렉션



계정 탈퇴

사용자 계정 및 관련 데이터 영구 삭제

비밀번호 재확인 → 계정 정보 삭제 → 할일 데이터 자동 삭제

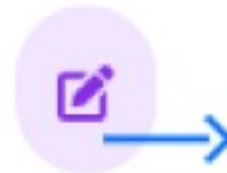
☰ 할일 관리 기능

직관적인 CRUD 작업을 통한 효율적인 태스크 관리

할일 관리 흐름



생성



수정



완료



삭제



생성 (Create)

- 새로운 할일 추가
- 제목, 내용, 카테고리, 우선순위, 마감일 설정



수정 (Update)

- 기존 할일 내용 변경
- 모든 속성 편집 가능



완료 토글 (Toggle)

- 클릭 한 번으로 완료/미완료 전환
- 시각적 완료 표시 (체크박스, 취소선)



삭제 (Delete)

- 불필요한 할일 목록에서 제거
-

Api 설계

인증 (Authentication)

POST /api/register

 신규 사용자 계정을 생성합니다.

POST /api/login

 사용자 로그인 및 JWT 토큰 발급을 처리합니다.

GET /api/me

 현재 로그인된 사용자 정보를 조회합니다.

POST /api/logout

 사용자 로그아웃을 처리합니다.

할일 관리 (Todo Management)

GET /api/todos

 사용자의 모든 할일 목록을 조회합니다.

POST /api/todos

 새로운 할일을 생성합니다.

PUT /api/todos/{id}

 특정 할일의 전체 내용을 수정합니다.

PATCH /api/todos/{id}/toggle

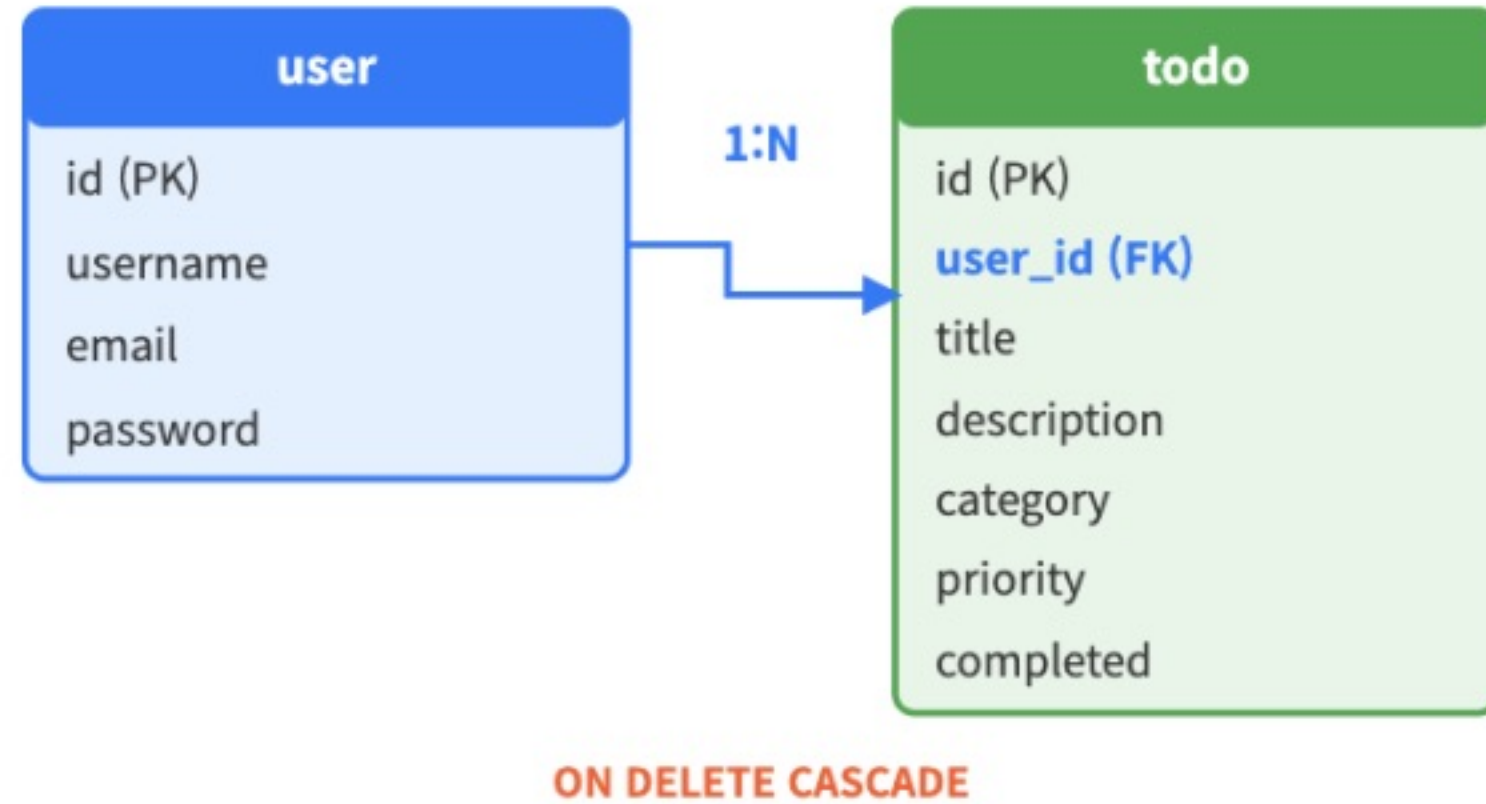
 특정 할일의 완료 상태를 변경합니다.

DELETE /api/todos/{id}

 특정 할일을 삭제합니다.

데이터베이스 설계

ERD 다이어그램



테이블 관계

user와 **todo** 테이블은 **1:N(일대다)** 관계를 가집니다.

- 한 명의 사용자는 여러 개의 할일을 가질 수 있음
- 각 할일은 반드시 한 명의 사용자에게 속함

외래키 및 제약조건

todo 테이블은 **user_id**를 외래키(Foreign Key)로 가지며, **user** 테이블의 **id**를 참조합니다.

```
FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
```

ON DELETE CASCADE 옵션: 사용자가 탈퇴할 경우 해당 사용자의 모든 할일 데이터가 함께 삭제됨

인덱스 최적화

자주 조회되는 컬럼에 인덱스를 적용하여 쿼리 성능을 최적화했습니다.

todo 테이블의 **user_id** 컬럼에 인덱스 적용

```
CREATE INDEX idx_todo_user_id ON todo(user_id);
```

→ 특정 사용자의 할일 목록을 조회하는 성능이 향상됨

에러 처리

에러 상황 분석

예측 가능한 에러

- ✓ 개발자가 미리 예상할 수 있는 상황
- ✓ 사용자 입력 검증 오류
- ✓ 비즈니스 로직 위반
- ✓ 리소스 존재 여부 확인
- ✓ **4xx** 상태 코드로 처리

예측 불가능한 에러

- ✓ 개발자가 예상하지 못한 상황
- ✓ 데이터베이스 연결 실패
- ✓ 외부 API 장애
- ✓ 시스템 레벨 오류
- ✓ **5xx** 상태 코드로 처리

HTTP 상태코드 분류

400 Bad Request

필수값 누락, 형식 오류

401 Unauthorized

인증 실패, 로그인 필요

403 Forbidden

권한 없음, 접근 거부

404 Not Found

리소스 없음, 존재하지 않는 데이터

409 Conflict

충돌, 이미 존재하는 리소스

500 Internal Server Error

서버 에러, DB 장애, 예상치 못한 오류

구현 전략

```
@app.put("/todos/{todo_id}")
def modify_todo(...):
    try:
        # 1. 예측 가능한 예외 처리
        ...
        raise HTTPException(...) # 400 or 404

        # 2. DB 수정 시도 (정상 흐름)
        ...

    except HTTPException:
        raise # 예측 가능한 에러는 그대로 응답

    except Exception as err:
        # 예측 불가능한 에러는 500 에러로 포장
        raise HTTPException(status_code=500, detail="서버 오류")
```

실제 적용

회원가입 API 에러 처리

예측 가능한 에러:

- 이메일 공백 여부 확인 → 400 에러
- 비밀번호 공백 여부 확인 → 400 에러
- 이름 공백 여부 확인 → 400 에러
- 이미 존재하는 이메일 확인 → 409 에러

예측 불가능한 에러:

- DB 연결 실패, 기타 시스템 오류 → 500 에러

할일 수정 API 에러 처리

예측 가능한 에러:

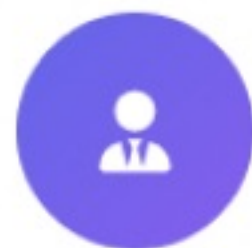
- 존재하지 않는 할일 ID → 404 에러
- 제목, 설명, 카테고리 공백 확인 → 400 에러
- done 값 타입 검증 (boolean) → 400 에러

예측 불가능한 에러:

- DB 업데이트 실패 → 500 에러

AI 활용 방법

AI 활용 1: 멘토처럼, 동료처럼, 직원처럼



멘토처럼

- ✓ 체계적인 학습 로드맵 제공
- ✓ 복잡한 개념을 쉽게 설명
- ✓ 코드 리뷰 및 피드백
- ✓ 실무 중심 미션 제공



동료처럼

- ✓ 막힌 문제 함께 해결
- ✓ 아이디어 브레인스토밍
- ✓ 기술적 의사결정 검토
- ✓ 실시간 질의응답



직원처럼

- ✓ 반복적인 코드 작성
- ✓ CRUD 기본 틀 생성
- ✓ 포매팅 및 문서화
- ✓ 변수명 제안 및 정리

💡 실제 활용 예시

"FastAPI + MySQL로 REST API를 만들려는데, 백엔드 기초도 모르겠어. 시니어 개발자라 생각하고 실무 중심의 학습 로드맵을 투두 형식으로 만들어줘."

"내가 작성한 이 코드를 시니어 개발자 입장에서 리뷰해주고, 개선점이 있다면 구체적으로 알려줘."

"이 CRUD API의 기본 틀을 만들어주고, 각 엔드포인트별 응답 형식도 정의해줘."

📌 균형 잡힌 학습 원칙

🔍 검증하는 습관

AI 답변을 공식문서와 다양한 자료로 교차 검증

💥 사고하는 힘

AI가 만든 코드의 동작 원리를 이해하려 노력

⚖️ 적절한 의존도

핵심은 직접, 반복작업은 AI에게 위임

🎓 지속적 성장

AI와 함께 사고하되, AI에만 의존하지 않기

AI 활용 2: 바이트 코딩을 활용한 개발

</> 바이트 코딩 개발 프로세스

1 요구사항 정의

- 기능 명세 작성
- UI 디자인

2 구체적인 프롬프팅

- 명확하고 구체적인 지시사항 작성
- 기대 결과물에 대한 사전 정의
- 기술적 제약 및 조건 명시

3 이미지 투입

- 참고 UI 스크린샷

4 반복 개선

- 결과물 검토
- 피드백 반영
- 점진적 완성

📈 성과 및 아쉬운점

🏆 주요 성과

- 명확한 요구사항 정의로 빠른 개발 결과 도출
- 시각 자료 활용으로 AI의 이해도 및 출력 품질 향상
- PyQt6 기초 지식만으로도 기능 구현에 성공
- 반복 개선을 통해 완성도 점진적 향상

⚠️ 아쉬운점

- PyQt6 경험 부족으로 인한 코드 검증의 어려움
- 파이썬 개발 경험 부족으로 제한된 프롬프트 작성
- 이미지 기반 입력 방식의 정확도 한계
- MCP 미활용
- 간단한 요구사항조차 해결이 어려운 경우 존재

✂️ 주요 도구



Cursor IDE

AI 페어 프로그래밍

🔑 향후 개선 방향

- MCP 활용을 통한 효율성 향상
- 더 정교한 AI 활용 전략 수립

"앞으로도 고민과 학습이 필요하지만, AI를 활용한 개발의 가능성을 분명히 확인했습니다."

프로젝트를 통해 성장한점



백엔드 실전 감각 향상

이번 프로젝트를 통해 **백엔드 개발을 처음부터 직접 경험**해보며, 서버와 데이터베이스의 동작 원리를 체감할 수 있었습니다.

- ✓ FastAPI로 REST API 설계 및 개발
- ✓ MariaDB + DBeaver로 데이터 관리 경험
- ✓ Docker로 개발 환경 구성 및 서비스 실행

→ 백엔드 전반에 대한 실전 감각과 자신감 확보



AI를 통한 주도적 학습

처음 접하는 기술도 **AI를 적극 활용**하며 막막함을 극복하고, 나만의 학습 루틴을 만들어갔습니다.

- ✓ AI에게 학습 순서, 로드맵, 개념 설명 요청
- ✓ 반복 작업은 위임, 핵심 로직은 직접 개발
- ✓ AI 조언 + 공식문서 + 스스로 검증의 균형

→ AI에 의존하지 않고 함께 성장하는 주도적인 학습 방식 확립



바이브 코딩을 통한 몰입형 개발

이번 프로젝트에서는 **바이브 코딩 방식**으로 프론트엔드 개발을 진행하며, 구현 흐름에 집중하고 맥락을 놓치지 않는 개발 루틴을 경험했습니다.

- ✓ 화면 구성과 코드 작성의 실시간 피드백
- ✓ UI 흐름을 눈으로 확인하며 빠르게 반복
- ✓ 디자인 → 구현 → 개선의 자연스러운 순환

→ 맥락 중심의 몰입형 개발 방식 습득과 생산성 향상

Smart하게 할일을, Smart Task Manager



직관적인 인터페이스

별도의 학습 없이 누구나 바로 사용할 수 있도록 직관적인 데스크톱 UI를 구현했습니다.



유연한 백엔드 아키텍처

FastAPI 기반의 경량 백엔드를 빠르게 설계하고 빠르게 구현하였습니다.



사용자 중심의 경험 설계

실제 사용자의 작업 흐름을 바탕으로, 효율성과 몰입감을 높이는 방향으로 UI/UX를 구성했습니다.

❤️ 감사합니다

지금까지 Smart Task Manager 프로젝트를 들어주셔서 진심으로 감사드립니다.
개발자로서 한 걸음 성장할 수 있었던 소중한 여정이었습니다.

💡 질문 있으시면 언제든지 편하게 말씀해 주세요!